

COMMON COURSE ASSIGNMENT REPORT

WeatherGPT — Conversational AI for Weather Forecasting, Alerts and Climate Information

*Problem Statement ID: WeatherGPT | Organization: Ministry of Earth Sciences (MoES) — India
Meteorological Department (IMD)*

Category: Software | Theme: Disaster Management

Field	Details to be filled in by student
Department	BTECH
Programme	AIML
Course Code & Course Name	CSA1008 – SOFTWARE ENGINEERING
Academic Year / Batch	2025
Faculty Name	DR. K. C. SHARMILI
Team / Student Name(s) & Roll No(s).	ARTHI A - 192525152
Date of Issue	3.9.2026
Date of Submission	4.9.2026
Maximum Marks	100

Table of Contents

Table of Contents	2
A. Assignment Information.....	3
B. Assignment Problem / Challenge.....	3
C. Problem Statement.....	3
D. Requirements and Constraints.....	4
D.1 Functional Requirements.....	4
D.2 Non-Functional Requirements.....	4
D.3 Constraints.....	4
E. Student Work.....	5
E.1 Problem Understanding and Formulation	5
E.2 Application of Course Knowledge	5
E.3 Solution / Design / Methodology	6
Diagrams to prepare (see Appendix A for the full draw.io checklist)	6
E.4 Use of Modern Tools.....	10
E.5 Results and Validation.....	10
E.6 Analysis and Engineering Decision.....	14
E.7 Broader Considerations	15
E.8 Conclusion	15
E.9 Student Reflection	15
E.10 References.....	15
F. Common Assessment Rubric (Institutional Format – 100 Marks)	16
G. WeatherGPT-Specific Rubric (100 Marks, per Assignment Guide)	Error! Bookmark not defined.
H. CO–PO–Assessment Mapping	16

A. Assignment Information

Particular	Details
Assignment Title	WeatherGPT — Conversational AI for Weather Forecasting, Alerts and Climate Information
Course Outcome(s) – CO	CO1–CO6 (Agile planning, requirements & design, full-stack development & collaboration, testing, responsible AI/emerging technology, end-to-end integration & demonstration)
Bloom's Taxonomy Level	L3 (Apply) to L6 (Create) — spanning requirement analysis, design, implementation, testing and evaluation
SDG Mapping (SDG 1-17)	SDG 2, SDG 3, SDG 9, SDG 11, SDG 13
Industry / Societal Relevance	Aligns with IMD/MoES's need for an accessible, AI-driven public weather-advisory channel that supports farmers, disaster-management teams, researchers and the general public — directly relevant to conversational-AI, GIS and disaster-management industry practice.

B. Assignment Problem / Challenge

This assignment requires students to apply software-engineering, requirements-engineering, UML/system-design and conversational-AI concepts to a realistic, agency-sponsored problem rather than reproduce theory. Students must:

- Identify the real-world weather-information problem faced by farmers, the public, researchers, disaster-management teams and government agencies.
- Analyse functional and non-functional requirements and prioritise them (MoSCoW).
- Consider technical, cost/time, data-availability, accessibility, multilingual and security/privacy constraints.
- Design and compare alternative architectures/technology choices for a conversational-AI weather platform.
- Use modern engineering/computing tools (Google Sheets, Jira, Confluence, Figma, Umbrello/draw.io, GitHub, Netlify/Vercel).
- Interpret test/deployment results and justify engineering decisions.
- Consider ethical, safety, environmental and societal factors relevant to disaster-management communication.

C. Problem Statement

Problem / Case / Design Challenge:

Design and develop “WeatherGPT,” an AI-powered conversational platform, sponsored by the Ministry of Earth Sciences (India Meteorological Department), that provides real-time weather information, forecasts, extreme-weather alerts, climate information and location-based advisories through natural-language interaction. The system must serve farmers, the general public, researchers, disaster-management teams and government agencies. The team is expected to: analyse requirements using Google Sheets and prioritise features with MoSCoW; manage the project using Jira; maintain documentation in Confluence; design the interface in Figma and the system architecture/UML in Umbrello (or draw.io, see Appendix A); build a working full-stack application supporting weather queries, location-based forecasts, alerts, multilingual support and historical/climate information; collaborate and version-control the code on GitHub (branches, commits, pull requests, reviews); and deploy the completed application on Netlify or Vercel, demonstrating it with realistic disaster-management scenarios.

D. Requirements and Constraints

D.1 Functional Requirements

- Accept natural-language weather queries (text, and voice where feasible) and return an accurate, conversational response.
- Provide location-based current weather and multi-day forecasts (GPS or manually entered location).
- Push extreme-weather alerts (cyclone, heavy rainfall, heatwave, etc.) for a selected location.
- Provide historical weather and climate-trend information (e.g., “how has rainfall in this district changed over 10 years?”).
- Support multiple Indian languages for query and response.
- Provide role-relevant advisories (e.g., crop-weather guidance for farmers, evacuation-relevant alerts for disaster-management teams).

D.2 Non-Functional Requirements

- Performance: forecast/alert responses returned within an acceptable response time (e.g., a few seconds).
- Reliability: graceful handling of third-party weather-API downtime or rate limits.
- Usability/accessibility: simple, low-literacy-friendly conversational interface.
- Security/privacy: safe handling of user location data; no unnecessary personal-data collection.
- Scalability: able to handle concurrent queries from multiple users/regions.

D.3 Constraints

- Technical: dependence on the accuracy/coverage of the underlying weather-data source or API.
- Cost/Time: academic-semester timeline and free/low-cost tool tiers (Jira, Confluence, Figma, Netlify/Vercel free plans).

- Data availability: access to a reliable historical/real-time weather dataset or API.
- Ethical/Responsible AI: avoiding false or exaggerated alerts; disclosing AI-generated advisory limitations.
- Standards: alignment with IMD alert-terminology conventions where applicable.

E. Student Work

E.1 Problem Understanding and Formulation

What is the problem?

Farmers, the public, researchers and disaster-management agencies currently need to consult multiple, technical weather sources; a single conversational interface that understands natural-language questions and returns clear, localized, multilingual and role-appropriate weather information/alerts is missing.

Expected outcomes:

- A deployed, working conversational-AI weather assistant.
- Demonstrated location-based forecasts, alerts, multilingual Q&A and historical/climate insights.
- Evidence of a complete Agile software-development lifecycle (requirements → design → build → test → deploy).

Information / data available:

- Public weather/climate APIs or datasets (e.g., IMD data portal, OpenWeatherMap, NASA POWER, or similar), to be identified and cited by the team.
- An LLM/NLP API or open-source model for natural-language understanding and generation.

Assumptions:

- Users have basic smartphone/internet access.
- A free-tier or academic-access weather API/LLM API is available for the team to use.

Constraints to satisfy:

As listed in Section D — functional, non-functional, cost/time, data-availability and responsible-AI constraints.

E.2 Application of Course Knowledge

The team should identify and explicitly apply relevant principles rather than merely describe the tools used, for example:

- Software Engineering / Agile principles: Scrum roles, sprint planning, backlog grooming, velocity tracking (via Jira).
- Requirements Engineering: functional vs non-functional classification, MoSCoW prioritisation (via Google Sheets).

- System/Software Design: UML modelling (use-case, class, sequence, deployment diagrams), layered/microservice architecture principles.
- Conversational AI / NLP concepts: intent recognition, entity extraction (location, date, weather parameter), prompt design, retrieval-augmented generation (RAG) if historical data is queried.
- Database design: schema for locations, forecast cache, alert subscriptions, and query logs; normalisation considerations.
- API design and integration: REST principles, authentication, rate-limit handling, caching of third-party responses.
- Software testing theory: unit, integration, usability and basic security testing.

Show the actual mapping — e.g., which MoSCoW category a requirement fell into and why, or which UML diagram models which subsystem — rather than only naming the concept.

E.3 Solution / Design / Methodology

Proposed high-level architecture (three-tier):

- Presentation layer: chat-style web front end (e.g., React/Vue), built and prototyped first in Figma.
- Application/API layer: backend service (e.g., Node.js/Express, Python/FastAPI or Flask) that receives the user message, calls the NLP/LLM component to interpret intent and location, and orchestrates calls to the weather-data API.
- Data layer: database (e.g., PostgreSQL/MongoDB) for caching forecasts, storing alert subscriptions and query history, plus the external weather/climate API/dataset as the source of truth.

Suggested request flow:

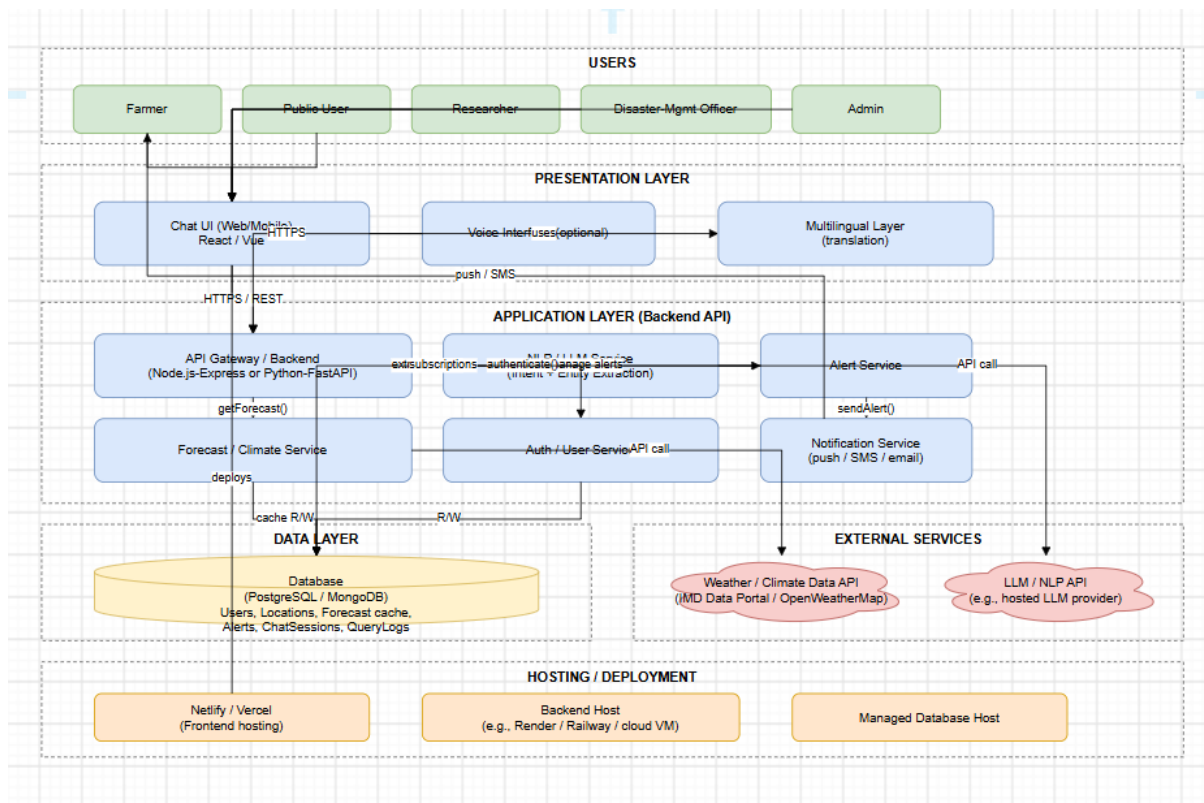
User message → NLP/LLM intent & entity extraction (location, weather parameter, time range) → backend orchestration → weather-API / historical-database lookup → response composition (incl. multilingual translation if needed) → conversational reply and/or alert push to the user.

Alternative designs to compare (Section E.6 asks for the justified choice):

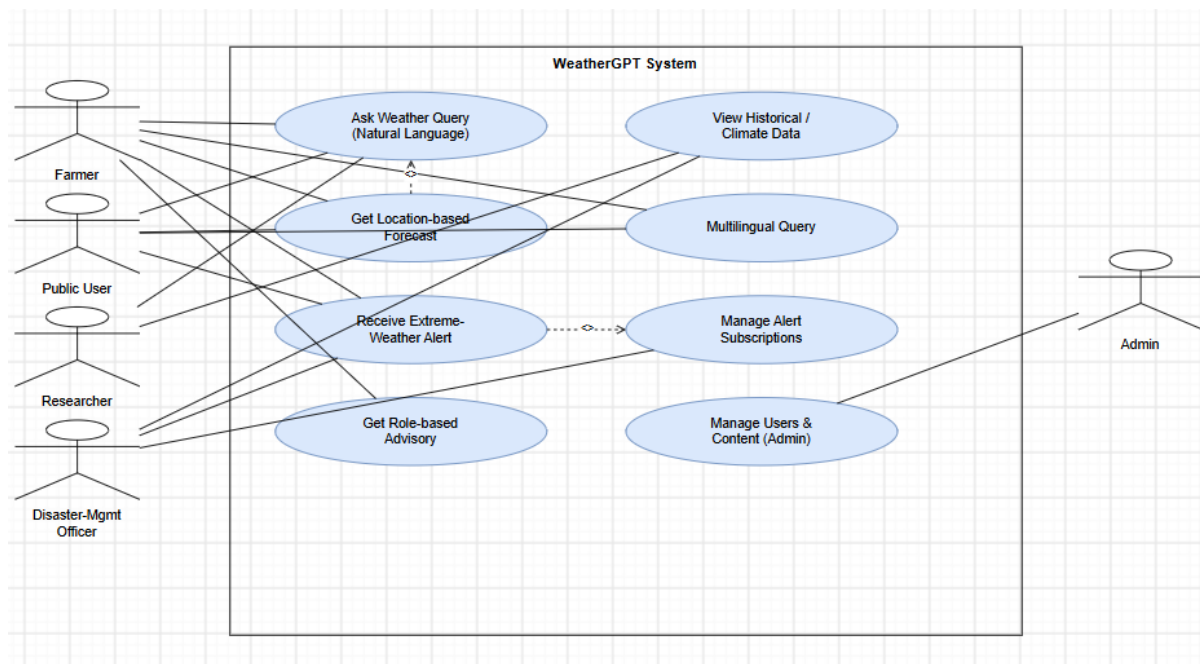
- Rule-based / intent-slot chatbot vs. LLM-based conversational engine.
- Monolithic backend vs. microservices (separate alert service, forecast service, chat service).
- Polling-based alert delivery vs. push-notification/webhook-based alert delivery.

Diagrams to prepare (see Appendix A for the full draw.io checklist)

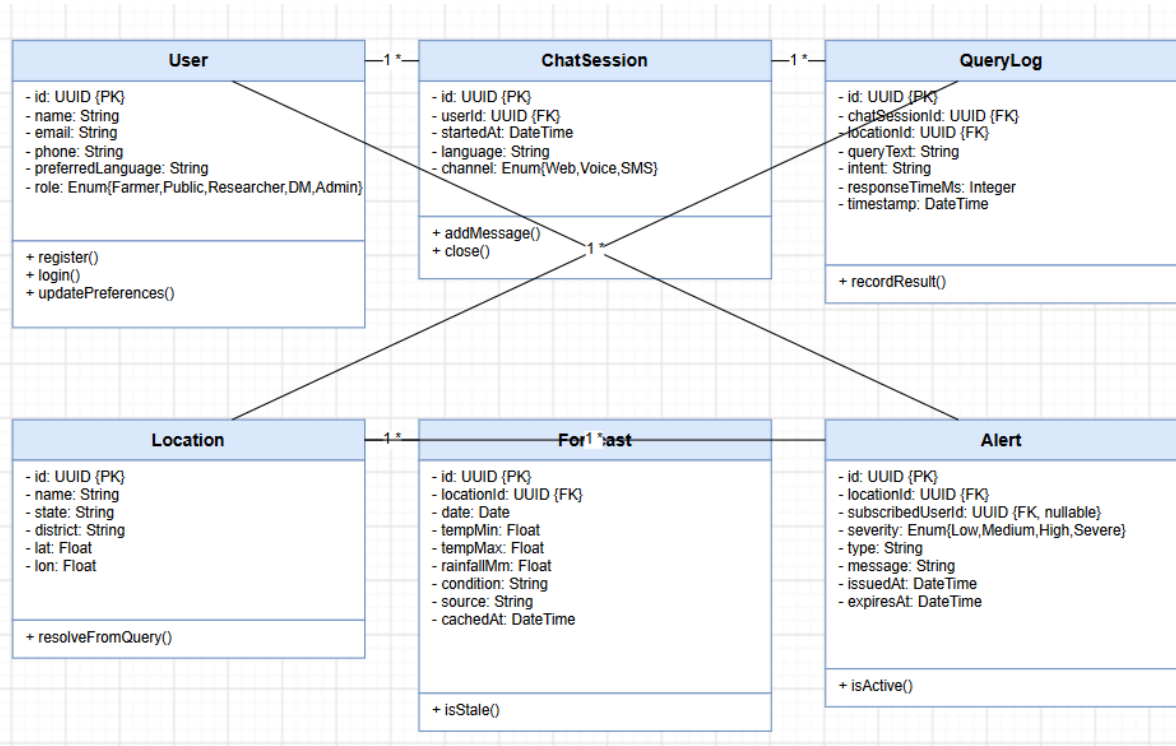
- System architecture diagram.



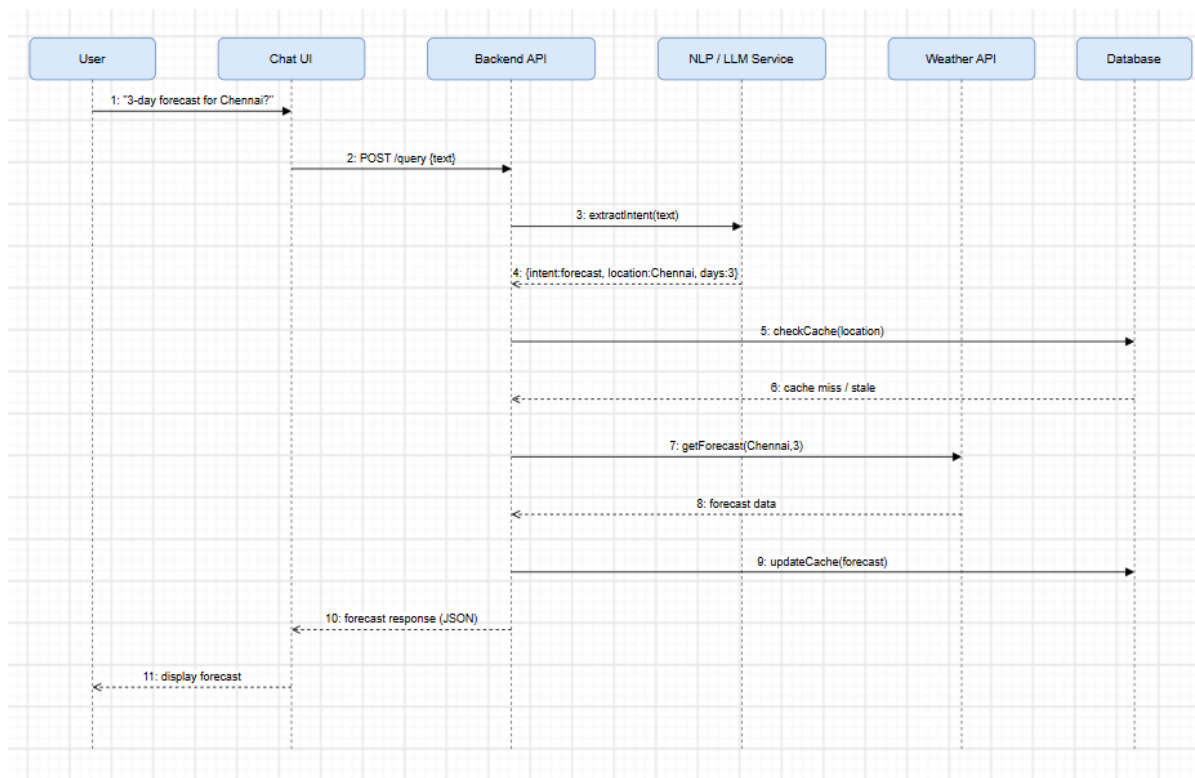
- UML use-case diagram (actors: Farmer, Public User, Researcher, Disaster-Management Officer, Admin).

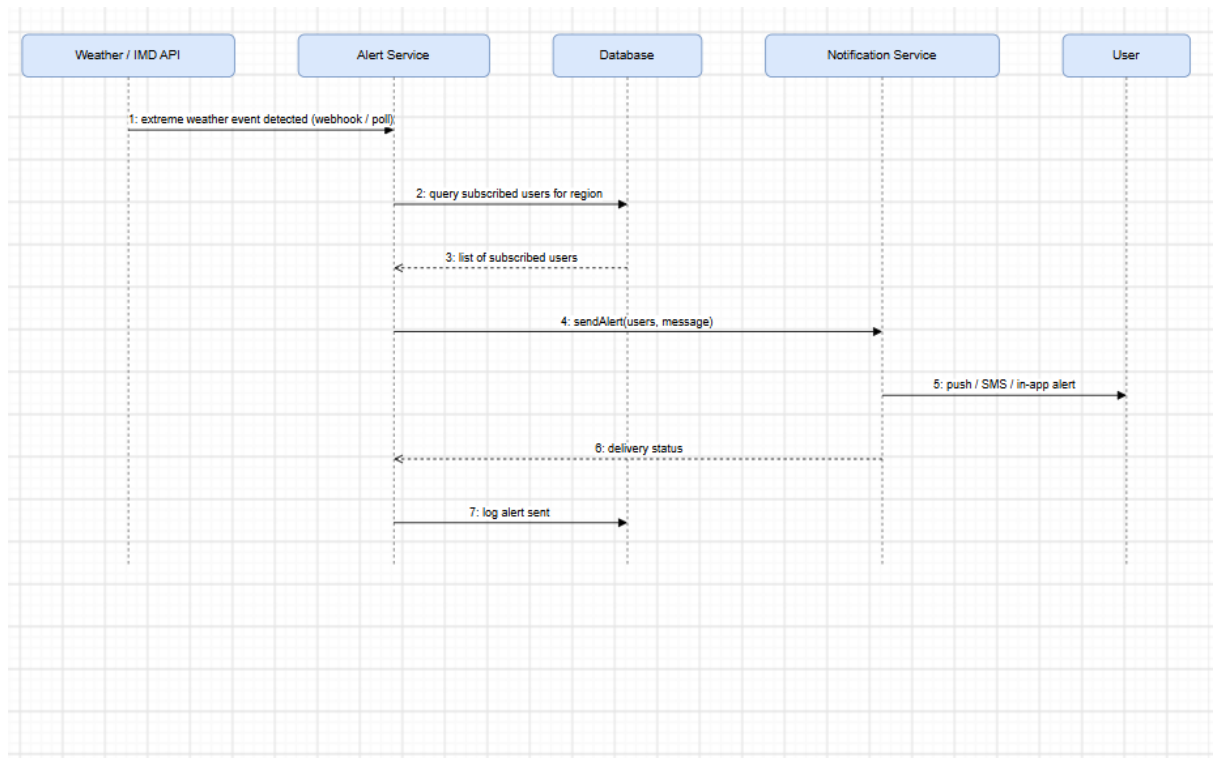


- UML class diagram (User, Location, Forecast, Alert, ChatSession, QueryLog entities).

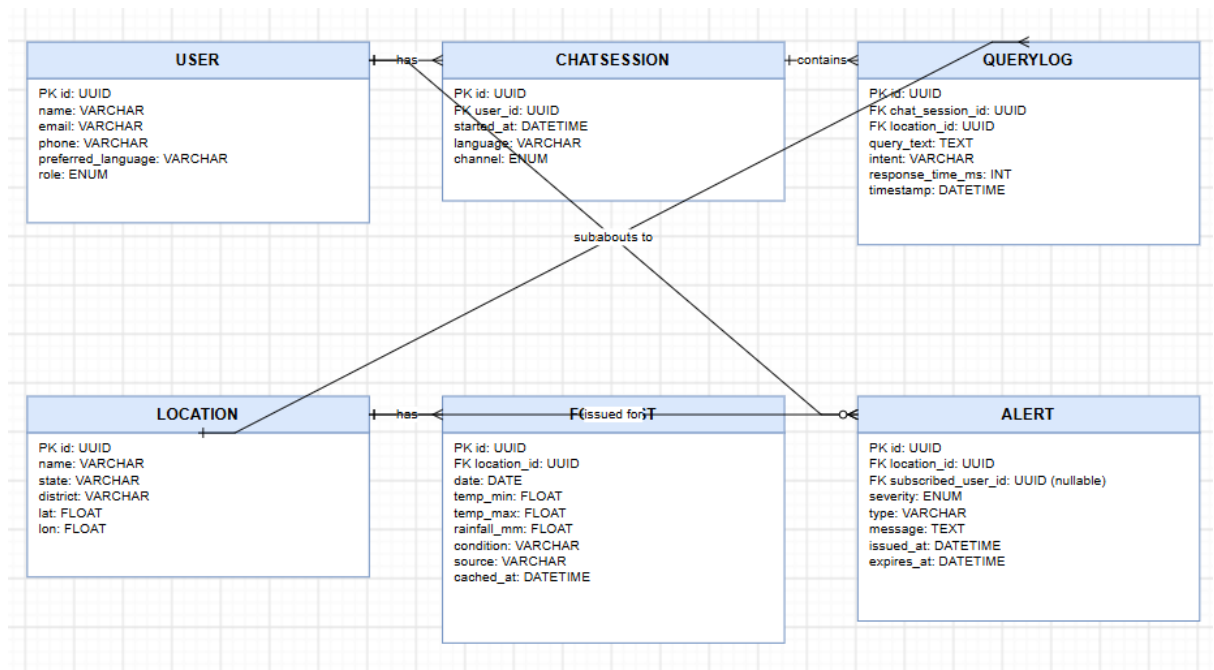


- Sequence diagram for “user asks for a 3-day forecast” and for “extreme-weather alert push.”

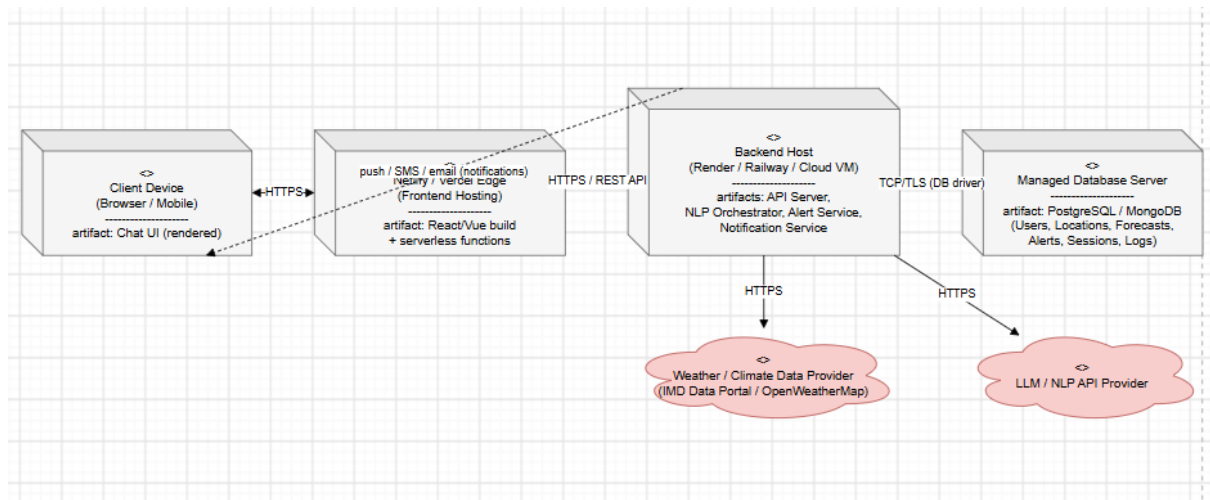




- ER diagram / database schema



- Deployment diagram (client, backend host, database, third-party weather/LLM APIs, Netlify/Vercel hosting).



E.4 Use of Modern Tools

Tool	Purpose in this project	Evidence to capture
Google Sheets	Requirements list, functional/non-functional classification, MoSCoW prioritisation	Screenshot of requirements sheet
Jira	Backlog, sprint planning, task allocation and progress tracking	Screenshot of backlog/board and burndown
Confluence	Project documentation and decision log	Screenshot of documentation pages
Figma	UI/UX design and interactive prototype	Screenshots of key screens / prototype link
Umbrello / draw.io	UML and system/architecture diagrams	Exported diagram images (see Appendix A)
GitHub	Version control: branches, commits, pull requests, code review	Screenshot of repo, commit history, PRs
Netlify / Vercel	Deployment of the working application	Screenshot of live deployed app + URL

E.5 Results and Validation

Present results with concrete evidence, for example:

- A test-case table covering: a normal-weather query, a forecast query, an alert-trigger scenario, a multilingual query, and an edge case (invalid/unclear location).
- Screenshots of the chatbot handling each of the above scenarios end-to-end.
- Basic performance observation (e.g., average response time for a query).
- Usability feedback from a small number of test users, if collected.

- Validation statement: confirm which functional/non-functional requirements from Section D were met, partially met, or not met, and why.

```
C:\Users\arthi\Downloads\files>npm install

added 68 packages, and audited 69 packages in 5s

15 packages are looking for funding
  run `npm fund` for details

3 moderate severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

```
C:\Users\arthi\Downloads\files>node server.js
WeatherGPT backend running on http://localhost:3000
█
```

 **WeatherGPT**
Conversational weather, forecasts & alerts

Hi! Ask me about current weather, a multi-day forecast, or active weather alerts for any location.

Try: "What's the weather in Chennai today?" · "3-day forecast for Coimbatore" · "any alerts near Chennai?"

Ask about the weather...

Send



WeatherGPT

Conversational weather, forecasts & alerts

Hi! Ask me about current weather, a multi-day forecast, or active weather alerts for any location.

What's the weather in Chennai today?

Current weather in Chennai: Light drizzle, 33°C, wind 9 km/h.

Try: "What's the weather in Chennai today?" · "3-day forecast for Coimbatore" · "any alerts near Chennai?"

Ask about the weather...

Send



WeatherGPT

Conversational weather, forecasts & alerts

Hi! Ask me about current weather, a multi-day forecast, or active weather alerts for any location.

What's the weather in Chennai today?

Current weather in Chennai: Light drizzle, 33°C, wind 9 km/h.

wether in banglor

Current weather in Banglor: Light drizzle, 23.9°C, wind 5.9 km/h.

Try: "What's the weather in Chennai today?" · "3-day forecast for Coimbatore" · "any alerts near Chennai?"

Ask about the weather...

Send

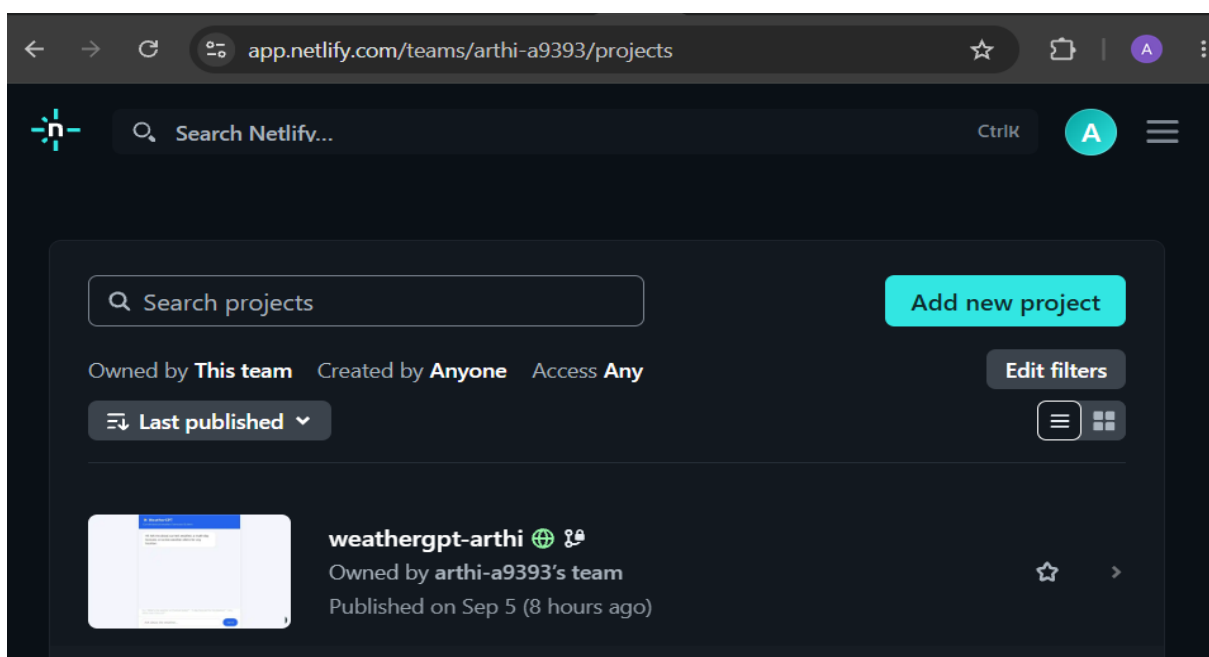
Test Case	Input	Expected Output	Result
TC1 – Current weather	“What’s the weather in Chennai today?”	Correct current conditions for Chennai	Pass/Fail
TC2 – Forecast	“3-day forecast for my location” (GPS)	3-day forecast for resolved location	Pass/Fail
TC3 – Alert	Simulated cyclone warning trigger	Alert pushed to subscribed users	Pass/Fail
TC4 – Multilingual	Query in a regional language	Correct response in same language	Pass/Fail
TC5 – Edge case	Ambiguous/misspelled location	Graceful clarification prompt	Pass/Fail

E.6 Analysis and Engineering Decision

Discuss the trade-offs actually faced, for example:

- LLM-API-based NLU vs. a custom-trained intent classifier — cost, accuracy, latency, offline capability.
- SQL vs. NoSQL database for forecast caching and query logs — schema flexibility vs. relational integrity.
- Netlify vs. Vercel for deployment — build-pipeline fit, backend/API-route support, free-tier limits.
- Push-based vs. poll-based alerting — timeliness vs. implementation complexity.

For each decision actually made, justify it with the quantitative (e.g., latency, cost) or qualitative (e.g., team familiarity, documentation quality) evidence collected while building the project.



E.7 Broader Considerations

- Sustainability / Climate (SDG 13): supports climate-trend awareness and adaptation planning.
- Society & Health (SDG 3): timely alerts can reduce weather-related health and safety risk.
- Food security (SDG 2): crop-relevant weather guidance supports farmer decision-making.
- Infrastructure/Innovation (SDG 9): responsible use of AI/LLM, APIs and cloud/GIS technologies.
- Resilient communities (SDG 11): strengthens disaster-preparedness communication.
- Ethics/Responsible AI: avoid false alarms or misinformation, disclose AI-generated content, avoid bias against low-connectivity or non-English-speaking users, protect location-privacy.

E.8 Conclusion

Summarise, once the project is built: the final solution actually implemented, the major findings (what worked, what didn't), which functional/non-functional requirements were achieved, known limitations (e.g., API coverage gaps, limited language support), and concrete possible improvements (e.g., offline mode, more languages, SMS-based alerts for low-connectivity users).

E.9 Student Reflection

Each team member should briefly answer (see the WeatherGPT guide's 8-point reflection list):

- What did we learn, beyond what was directly taught?
- Why did we choose our UI, architecture and technologies?
- What challenges did we face, and how did we solve them?
- What did each member contribute?
- How did our work achieve CO1–CO6?
- Which SDGs does WeatherGPT support, and how?
- What can be improved in future?

E.10 References

Cite, in a consistent format (e.g., IEEE or APA), all external sources, datasets, standards, software/APIs and AI-assisted tools used — for example the weather/climate data source, any LLM/NLP API or model, and any open-source libraries used. Example placeholders:

- [1] India Meteorological Department, “Data/API source used,” [Online]. Available: <insert URL>.
- [2] <LLM/NLP API or model used>, provider documentation, <insert URL>.
- [3] Relevant open-source library/framework documentation used in implementation.

F. Common Assessment Rubric (Institutional Format – 100 Marks)

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	Bloom's Level	Marks
Problem formulation			10
Application of knowledge			20
Solution / Design / Implementation			20
Modern tool usage			10
Validation			15
Analysis & justification			15
Broader considerations			5
Documentation & reflection			5

Note: only the POs and Bloom's levels genuinely assessed by this assignment should be mapped by faculty; this table follows the institutional note that not every assignment maps to every PO.

Appendix

1. GITHUB LINK - <https://github.com/arthi9393/CSA1008---SOFTWARE-ENGINEERING/tree/main/PROJECT%20-%20ASSIGNMENT>
2. NETLIFY DEPLOYMENT LINK - weathergpt-arthi.netlify.app